Client Codebase A-to-Z Audit Report

Tech Stack: React 18, Material UI 5, Emotion, Vite, Axios, React Router 6
Source: `client/src/` — 63 page files, ~30 components, 7 contexts, 6 hooks, 23+ services/utils
Audit Scope: DRY, Maintainability, Security, Performance, UI/UX, Hardcoding, Architecture

A — API Architecture (Services Layer)

❌ Inconsistent service layer usage

- **7+ admin pages** bypass service files and call `api.*` directly (EnquiriesPage, SeoOverridesPage, CustomersPage, ShippingPage, SettingsPage, FeaturesPage, PaymentGatewaysPage, ApiBuilderPage)
- Mix of patterns: some use named exports, some default objects, one class-based (`MenuService`)
- No standard response extraction — callers guess between `response.data` , `response.data.data` , `response.data.data.tokens`

❌ No request deduplication or retry logic

- No retry for 5xx, network timeouts, or 429 rate limits
- No request cancellation (only `mediaService.js` accepts `AbortSignal`)
- `Intl.NumberFormat` recreated on every render in `useSettings.js:73` (not wrapped in `useMemo`)

❌ `api.js` — hardcoded localhost fallback

```
baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api'
```

B — Bundle & Build

⚠️ Vite config (`vite.config.js`)

- Sourcemaps disabled in production ✅
- No code-splitting strategy beyond route-level `React.lazy`
- Proxy settings for `/api` and `/uploads` are dev-only, no production proxy config

C — Component Design

❌ Monolithic components

File	Lines	Issue
`admin/ProductEditPage.jsx`	~2300	Single file, 30+ form fields in one state
`admin/SettingsPage.jsx`	~1200	Multiple settings panels in one component
`admin/OrderDetailPage.jsx`	~1500	All order detail logic inline
`admin/ApiBuilderPage.jsx`	1067	Full CRUD + code preview inline
`storefront/CheckoutPage.jsx`	~600	Multi-step form, payment, address in one file
`layouts/AdminLayout.jsx`	707	Menu + drag-drop + search + favorites + keyboard shortcuts
`layouts/StoreLayout.jsx`	532	3 menu types + announcement bar + header + mobile + account

❌ Missing React.memo on pure components

- `StatusBadge`, `SectionLabel`, `MetaCard`, `CenteredLoader`, `StatCard`, `ShareButton`

❌ Inline functions/objects recreated every render

- Massive inline `sx` objects in `ProductCard.jsx:46-214`, `OrderRow.jsx:17-132`
- `renderCell` in `ProductsManagePage.jsx:482-695` recreates components on each render

D — DRY Violations

🔴 High-Priority Duplications

Pattern	Locations	Count
Scroll-left/right detection	`CategoryNav.jsx`, `ProductRow.jsx`, `BrandStrip.jsx`, `SearchWidget.jsx`	4x
`getTaxRows`	`OrderDetailPage.jsx`, `OrderInvoicePage.jsx` (storefront + admin)	3x
`generateSlug`	`BrandsPage.jsx`, `ProductEditPage.jsx`	2x
`toDateTimeLocal`	`ProductEditPage.jsx`, `ProductsManagePage.jsx`, `SaleLabelsPage.jsx`	3x
Category tree flattening	`CategoriesPage.jsx`, `ProductsManagePage.jsx`, `CouponsPage.jsx`, `ApiBuilderPage.jsx`	4x
Tax calculation logic	`CheckoutPage.jsx`, `CartPage.jsx`	2x
`formatDateOnly`	`DeliverySummaryCard.jsx`, `ProductTrackingCard.jsx`	2x
`isExternalUrl`	`StorefrontFooter.jsx`, `StorefrontSidebarMenu.jsx`	2x
`SummaryCard` wrapper	`ProductsManagePage.jsx`, `OrdersManagePage.jsx`, `ReviewsPage.jsx`	3x
`TabPanel` component	`AccountPage.jsx`, `ShippingPage.jsx`	2x
`DetailCard` wrapper	`OrderDetailPage.jsx`, `CustomerDetailPage.jsx`	2x
URLSearchParams filter pattern	`adminService.js`, `attributeService.js`, `mediaService.js`, `wishlistService.js`	6x
`loadScript` (payment SDK)	`CheckoutPage.jsx`, `PaymentPage.jsx`	2x
Order status checks	`utils/orderWorkflow.js`, `hooks/useOrderStatusTransitions.js`	2x
Auth fetch-after-login	`AuthContext.jsx:69-72` (login), `:82-85` (register)	2x
`X-Session-Id` header	`cartService.js` — repeated in every method (5x)	5x

🟡 Duplicate page implementation

- `storefront/AllOrdersPage/AllOrdersPage.jsx` (906 lines) — **DEAD CODE**, replaced by `index.jsx` (491 lines) but never deleted

E — Error Handling

❌ `console.error` in production (9+ files)

Used instead of user-facing notifications: `ReviewSection.jsx`, `MediaPicker.jsx`, `CategoryAttributesDialog.jsx`, `ProductCustomTabs.jsx`, `SEO.jsx`, `StaticPageView.jsx`, `EnquiriesPage.jsx`, `BrandStrip.jsx`, `ProductComboBuilder.jsx`

❌ Inconsistent error patterns

- Some components show user-friendly snackbars; others silently eat errors
- Services have no standard error handling — one catches and returns null, another propagates raw

---

F — File Organization

❌ Mixed export conventions

- Services use 5 different export patterns (named, default object, class, named+default, default only)
- `context/ThemeContext.jsx` exports `SettingsContext` — file name does not match export name
- `useSettings.js` imports from `'../context/ThemeContext'` — confusing indirection

❌ Empty/stub files

- `utils/payu.js` — completely empty (0 lines)
- `client/src/theme/` — empty directory

G — Git & History

❌ Dead code not removed

- `AllOrdersPage/AllOrdersPage.jsx` — old 906-line version left alongside refactored `index.jsx`
- Multiple commented-out code blocks (customer email cell, reply icon button, FormHelperText)

❌ Stray comments in production code

- `App.jsx:40` — `// here i am working`

H — Hooks & State Management

✅ Good patterns

- `useDebounce.js` — clean implementation
- `useAuth.js` — proper context guard
- `useSettings.js` — well-documented with JSDoc

❌ `useCurrency` — Intl.NumberFormat recreated every render

`hooks/useSettings.js:73` — `new Intl.NumberFormat()` inside hook body, not memoized

❌ Massive single state objects

- `ProductEditPage.jsx:197-233` — single `formData` with 30+ fields causes full re-render on every keystroke
- `CheckoutPage.jsx:261` — nested form as single state object

❌ `useEffect` dependency issues (suppressed lint warnings in 4+ files)

- `ProductDetailPage.jsx:105-107`, `ProductListPage.jsx:65-66`, `BrandDetailPage.jsx:74`, `CategoriesPage.jsx:485-487`

I — i18n & Locale

❌ `'en-US'` hardcoded in 4+ locations

- `DeliverySummaryCard.jsx:15`, `ProductTrackingCard.jsx:22`, `orderDetailUtils.js:69,77`, `SalesChart.jsx:18`
- `orderHelpers.js:27` uses `'en-IN'` — inconsistent
- `useSettings.js:71` falls back to `'USD'` for India-focused app

J — JavaScript Practices

❌ `document.execCommand` (deprecated/removed)

- `ProductCustomTabs.jsx:133` — uses deprecated `execCommand`, plus `window.prompt()` and `alert()` for URL input

❌ Direct DOM manipulation in React

- `RichTextEditor.jsx:60,64` — `innerHTML = ''`, `appendChild` outside React tree
- `ThemeContext.jsx:73-108` — `applyDocumentSettings` mutates `<head>` directly

❌ CSS-in-JS `<style>` tag injection

- `ProductTabsAccordion.jsx:229-306` — 70+ lines of CSS injected as `<style>` tag on every mount

---

K — Keys & Lists

✅ Skeleton keys `key={i}` acceptable for static loaders

✅ Admin tables use proper `rowId` / `product.id` keys

L — Loading & Empty States

❌ Missing proper loading states

- `AdminLoginPage` — only text "Authenticating..."
- `RegisterPage` — "Creating account..."
- `LoginPage` — "Logging in..."
- `ForgotPasswordPage` — "Sending..."
- `SeoOverridesPage` — just "Loading..." text in table cell

❌ Missing empty states

- Components without skeleton/placeholder state while fetching

---

M — Maintainability

❌ 15+ files exceed 300 lines (see Section C)

❌ Deep call chains in `utils/variantPricing.js` (4+ levels deep)

❌ `utils/orderWorkflow.js:111-195` — 84-line function with 10+ boolean flags, high cyclomatic complexity

❌ `AdminLayout.jsx:67-131` — inline `MENU_STRUCTURE` should be extracted

❌ Unused imports found in `ProductImages.jsx`, `ProductTabsAccordion.jsx`

N — Naming Conventions

❌ `isAdmin` checks for both `admin` AND `super_admin` (`AdminLoginPage.jsx:61`) — misleading

❌ Inconsistent variable naming: `rows` vs `items` vs `orders` for same pattern

❌ `context/ThemeContext.jsx` → exports `SettingsContext`

❌ `COLS_MAP` uses `2.4` (non-standard grid value) in `BrandsPage.jsx:20-25`

O — Optimization

❌ No data caching strategy

- `BrandContext` , `CategoryContext` , `CartContext` , `WishlistContext` all fetch on every mount
- No SWR/React Query/TanStack Query
- `MediaPicker.jsx:47-49` — module-level cache is mutable, shared across instances (race condition)

❌ No `prefers-reduced-motion` respect

- CSS transitions in `ProductCard.jsx` , `ProductImages.jsx` , etc. never check user preference

❌ `useEffect` refetches on sort change without debounce (`MediaPicker.jsx:93-97`)

P — Performance

❌ `JSON.stringify` in `useEffect` dependency (`VariantSelector.jsx:37-57`)

❌ `CheckoutPage.jsx:398` — `useEffect` with 11 dependencies, extremely fragile

❌ `BrandsPage.jsx` — 7 separate `useMemo` calls that could be combined

❌ Double fetch on login: login response returns user, then immediately calls `getMe()` (`AuthContext.jsx:67-69`)

Q — Query Parameters

✅ `axios` interceptor handles auth headers cleanly

❌ No CSRF tokens visible in any request

❌ Email verification token exposed as URL query parameter (`authService.js:47`)

R — Routing

✅ Good: `ProtectedRoute` with permission checking

✅ Good: `FeatureRoute` and `ModeRoute` guards

✅ Good: `React.lazy` + `Suspense` for code splitting

✅ Good: Duplicate route `product/:slug` and `products/:slug` (graceful handling)

❌ `PaymentFailurePage.jsx:9` uses `new URLSearchParams(window.location.search)` instead of `useLocation()`

S — Security

🔴 CRITICAL: XSS via unsanitized `dangerouslySetInnerHTML`

- `StaticPageView.jsx:125` — CMS page content rendered **WITHOUT** DOMPurify sanitization

```
dangerouslySetInnerHTML={{ __html: page.content }}  // NO SANITIZATION
```

🔴 CRITICAL: Email verification token in URL query parameter

- `authService.js:47` — `api.get('/auth/verify-email?token=${token}')` exposed to server logs, browser history, Referer header

🔴 HIGH: SMTP credentials leaked to client

- `MessagingSettingsPanel.jsx:105-108,175-176,183-185` — SMTP password, Twilio SID, Auth Token bound to form `value` — round-trip to browser as plaintext

🔴 HIGH: Client-side-only sensitive data masking

- `AuditLogPage.jsx:459-466` — masks `password`, `token`, `secret` in the browser after data is received; anyone inspecting API responses sees raw values

🟡 `localStorage` token storage — if XSS exists, all tokens compromised

🟡 No `X-Content-Type-Options: nosniff` / CSP headers visible

🟡 Guest session key `X-Session-Id` in localStorage — session hijackable via XSS

T — Testing

⚠️ No client-side test files found

- `client/package.json` has no test scripts or test dependencies
- No Jest/Vitest/Cypress/Playwright config

U — UI/UX

❌ Accessibility gaps

- Multiple `IconButton` components missing `aria-label`
- `BrandStrip.jsx:127-145` — chevron buttons at 34x34 below 44x44 touch target
- `ProductCard.jsx:210-212` — wishlist button at 34x34
- `ProductImages.jsx:150` — `rgba(0,0,0,0.58)` hover label may have insufficient contrast
- `aria-label={placeholder}` pattern in `ProductCustomTabs.jsx:251` — fragile

❌ Native blocking dialogs in production

- `ProductCustomTabs.jsx:108,111` — `alert()` and `prompt()` for link insertion (poor UX, blocked by modern browsers)

❌ Hardcoded store name in payment flows

- `CheckoutPage.jsx:471` — `name: 'My Store'`
- `PaymentPage.jsx:137` — `name: "My Store"`
- `ProductDetailPage.jsx:735,738` — `'My Store'`

❌ No dark mode support in brand strip

- `BrandStrip.jsx:54,84` — `rgba(255, 252, 246, 0.92)` background

V — Variables (Unused/Dead)

❌ Dead code

- `AllOrdersPage/AllOrdersPage.jsx` — 906 lines, superseded by `index.jsx`
- Commented code in `OrdersManagePage.jsx`, `EnquiriesPage.jsx`, `ProductEditPage.jsx`
- `utils/payu.js` — empty file

❌ Unused imports

- `ProductImages.jsx:2` — `Grid` imported but not used
- `ProductTabsAccordion.jsx:21` — `useTheme` imported globally AND inside Skeleton component

W — Webpack/Vite Config

⚠️ `vite.config.js` — minimal config

- Single plugin (`@vitejs/plugin-react`)
- No `define` globals, no manual chunks, no polyfill configuration
- Dev proxy only — no production config

X — XSS Vulnerabilities (see Section S)

Y — Why It Matters (Impact Summary)

Category	Count	Severity
Security (Critical)	3	🔴 Immediate action
DRY Violations	16+ patterns	🟡 Refactor sprint
Monolithic Components	7+ files >500 lines	🟡 Refactor sprint
Performance Issues	10+	🟡 Optimization pass
Hardcoded Strings	25+ locations	🟢 Low effort wins
Missing Loading States	6 pages	🟢 Quick wins
i18n/Locale Hardcoded	5 locations	🟢 Low effort
Dead Code	4 instances	🟢 Cleanup

Z — Zero-Day Action Items (Top Priority)

1. 🔴 **Fix XSS in** `StaticPageView.jsx:125` — Add `DOMPurify.sanitize()` before `dangerouslySetInnerHTML`
2. 🔴 **Fix email verification token exposure** — Use `POST` with body instead of URL query param
3. 🔴 **Remove credential round-trip** — Don't bind SMTP passwords / Twilio tokens to form values
4. 🟡 **Extract shared** `useScrollState` **hook** — Eliminates 4x duplicated scroll detection
5. 🟡 **Delete dead** `AllOrdersPage/AllOrdersPage.jsx` — 906 lines of dead code
6. 🟡 **Move inline** `api.*` **calls to service files** — 7+ pages bypass the service layer
7. 🟡 **Add** `React.memo` **to pure components** — `StatusBadge`, `SectionLabel`, `StatCard`, etc.
8. 🟢 **Extract duplicate utility functions** — `generateSlug`, `toDateTimeLocal`, `getTaxRows`, `formatDateOnly`
9. 🟢 **Centralize hardcoded strings** — Store name, locale, fallback URLs into constants/env
10. 🟢 **Add client-side tests** — Vitest + React Testing Library setup needed